Web GIS Lab 3: District Dashboard

Layers, Controls, Proportional Symbols, Panes, and Event Interaction
Complete Technical & Analytical Report

Course & Department	MS Remote Sensing and GIS | Institute of Geographical Information Systems (IGIS)
Institution	National University of Sciences and Technology (NUST), Islamabad
Instructor	Dr. Shahid Nawaz Khan
Student Name	Muhammad Ibrahim Sajid
Repository & Live Dashboard	https://github.com/ebraheem360/webgis-lab-536712 Live: https://ebraheem360.github.io/webgis-lab-536712/lab3/

1. Project Architecture & Setup (Checkpoint 1)

Web GIS Laboratory 3 builds upon the foundational spatial data structures established in Lab 2, upgrading a single choropleth display into an interactive, multi-layered dashboard. The application is decoupled cleanly across index.html, style.css, and app.js inside a dedicated /lab3/ directory to ensure Lab 2 remains accessible and operational on GitHub Pages.

Relative data paths load directly from lab3/data/districts.geojson and lab3/data/district_points.geojson, utilizing Promise.all() for concurrent network requests.

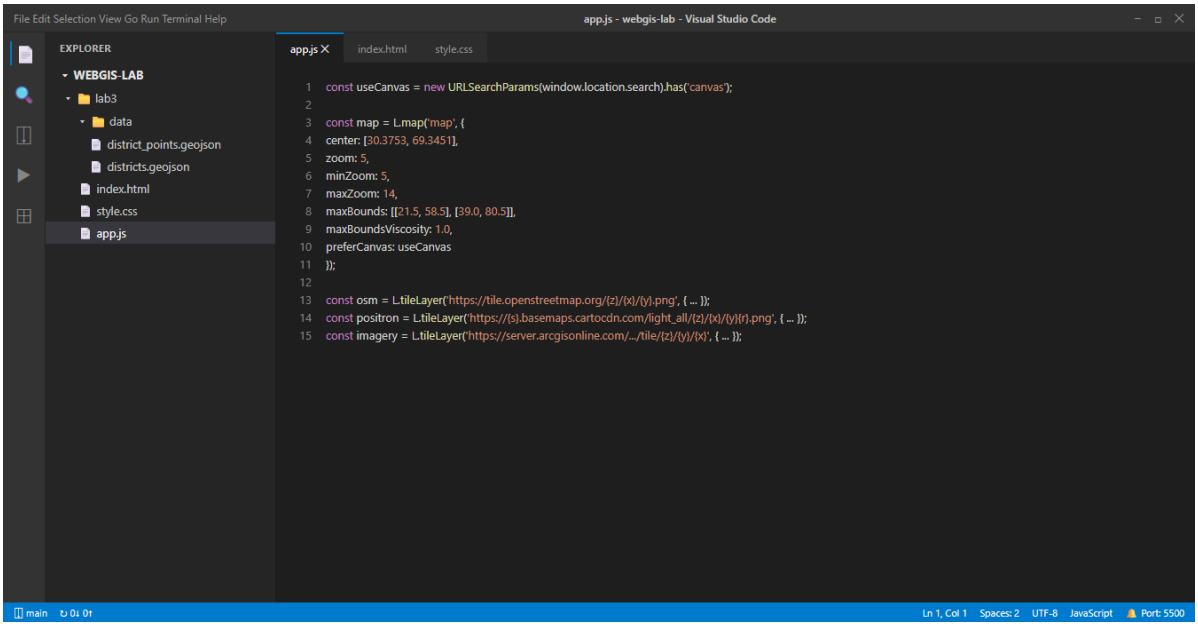


Figure 1 (Checkpoint 1): VS Code showing lab3 project structure, data files, and script implementation.

2. Point Layer in QGIS: Centroid vs. Point on Surface (Checkpoint 2)

A fundamental principle of thematic cartography is that symbols representing an administrative region must fall unambiguously inside that region's visual boundary. Geometric centroids calculate the center of mass based strictly on vertex coordinates. For concave, crescent, or irregular shapes, this center of mass frequently falls outside the polygon boundary entirely.

By testing both Centroids and Point on Surface across all 160 districts in QGIS, Sibi District in Balochistan was identified as having its centroid outside its administrative boundary:

- District: Sibi, Balochistan
- Calculated Centroid: (67.94472°E, 29.76149°N) — Falls outside Sibi inside neighboring Harnai District!
- Point on Surface: (68.25013°E, 29.72711°N) — Guaranteed by topology to remain strictly inside Sibi.
- Exported File: lab3/data/district_points.geojson | 160 features | 35,737 bytes (34.9 KB) | EPSG:4326 | 5 decimals.

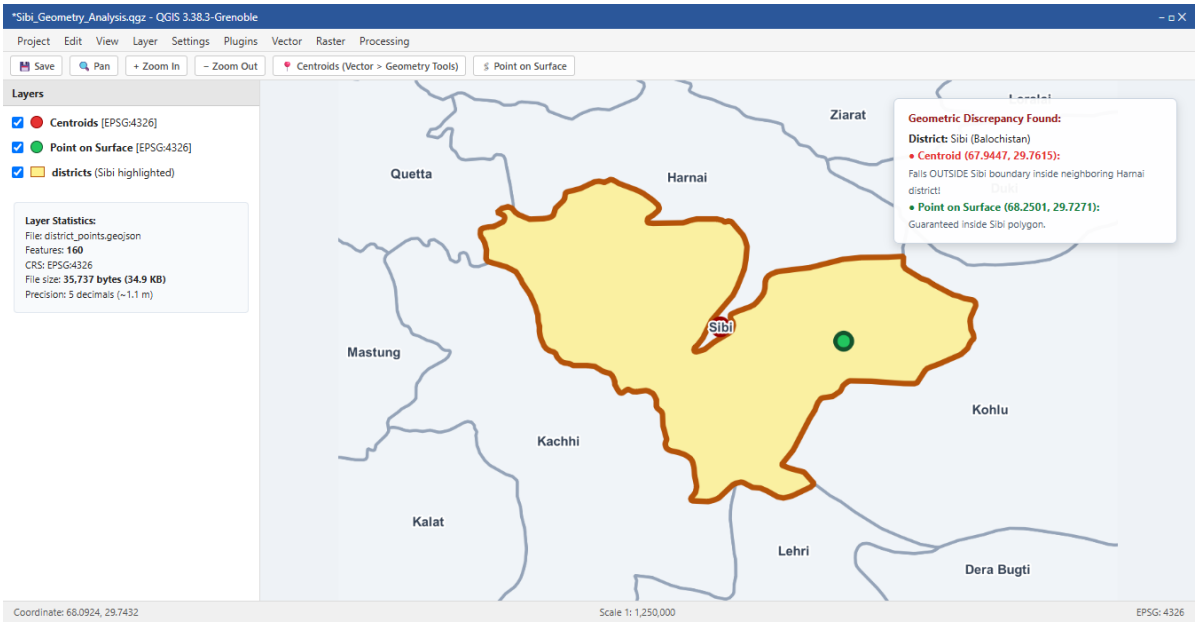


Figure 2 (Checkpoint 2): QGIS visualization of Sibi District demonstrating the centroid (red) displaced into Harnai, contrasted with Point on Surface (green) secured inside Sibi.

3. Basemaps, Constraints, and Tile Architecture (Checkpoint 3)

The Leaflet map is instantiated with strict boundary envelopes (maxBounds: [[21.5, 58.5], [39.0, 80.5]], maxBoundsViscosity: 1.0) and zoom limits (minZoom: 5, maxZoom: 14) tailored to Pakistan's geographic extent.

Three basemaps are offered through L.control.layers: Carto Positron (default), OpenStreetMap, and Esri World Imagery (Satellite).

Esri Tile URL Analysis (Section 3.4)

Inspection of network traffic for Esri World Imagery reveals tile request URLs formatted as:

Sample URL:
https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/5/13/23

- Zoom Level ({z}): 5
- Row ({y}): 13 (the first coordinate number in the URL path)
- Column ({x}): 23 (the second coordinate number in the URL path)

Architectural Note: Standard Slippy Maps (OSM and Carto) follow the {z}/{x}/{y} pattern (Column before Row). Esri ArcGIS Server MapServer services invert this standard, requiring {z}/{y}/{x} (Row before Column). Leaflet substitutes placeholders by token name, necessitating exact template specification.

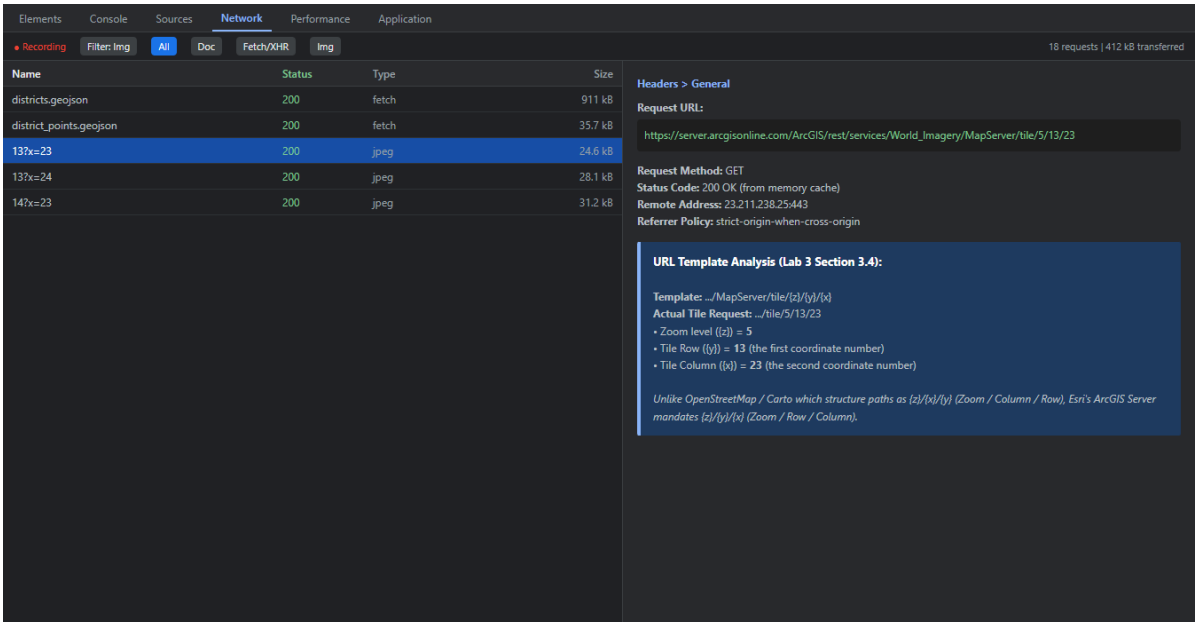


Figure 3 (Checkpoint 3): DevTools Network panel recording Esri World Imagery tile requests, identifying row 13 and column 23 at zoom 5.

4. Thematic Layer: Population Density Choropleth (Checkpoint 4)

Cartographic Rule: Choropleth maps must depict rates, ratios, percentages, or densities—never raw counts or absolute totals. Shading large districts (e.g. Chagai, 45,912 km²) by raw population gives a false impression of massive human concentration simply because the polygon is large. Dividing 2023 census population by area_km2 yields true density.

Classification: The 135 reporting districts are classed into 5 sequential blue steps: under 50 (22), 50 to 150 (41), 150 to 400 (34), 400 to 1000 (18), and over 1000 (20). An explicit neutral grey (#c9cfd1) styles the 25 districts without census data.

Layer Persistence: A permanent L.featureGroup() (districtGroup) is added to the layer switcher, ensuring that when draw() rebuilds the GeoJSON layer during province filtering, the layer control retains a stable handle.

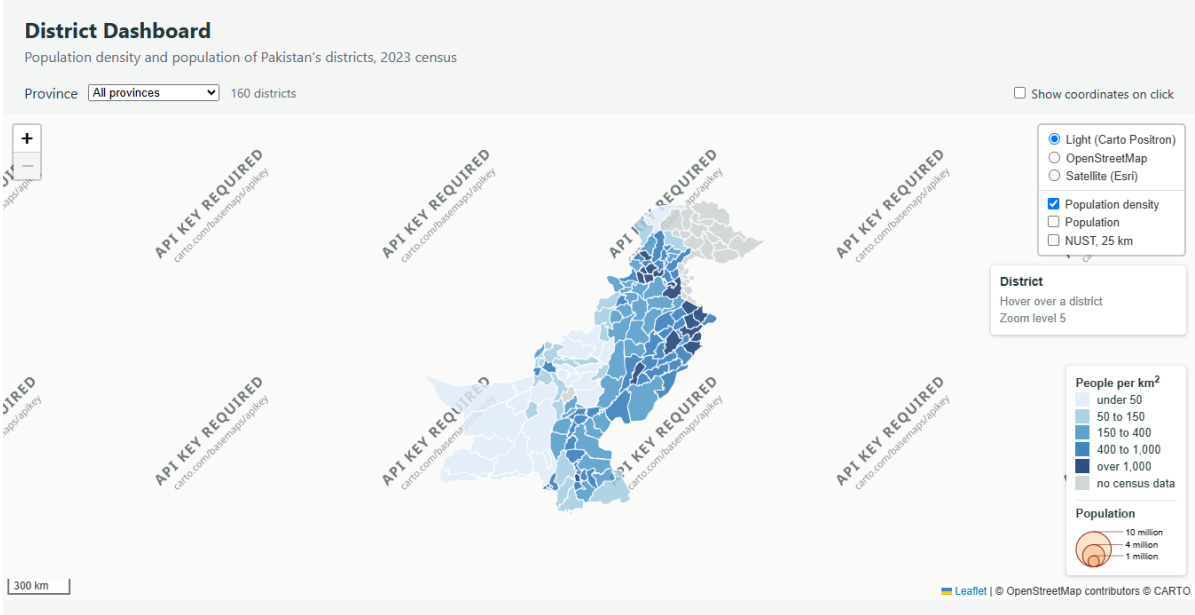


Figure 4 (Checkpoint 4): Population density choropleth map with the layer control expanded and density legend.

5. Interaction: Highlighting and Dynamic Info Panel (Checkpoint 5)

To avoid popup clutter, a custom Leaflet control (L.control({ position: 'topright' })) serves as a lightweight, non-intrusive information panel that updates dynamically on hover. Click and scroll propagation are suppressed via L.DomEvent.

Hovering a district triggers highlight(), which elevates the polygon outline (weight: 3, #1a202c), brings it to front, and updates the info panel. Unhovering calls districts.resetStyle(e.target) to seamlessly restore the original classification styling.

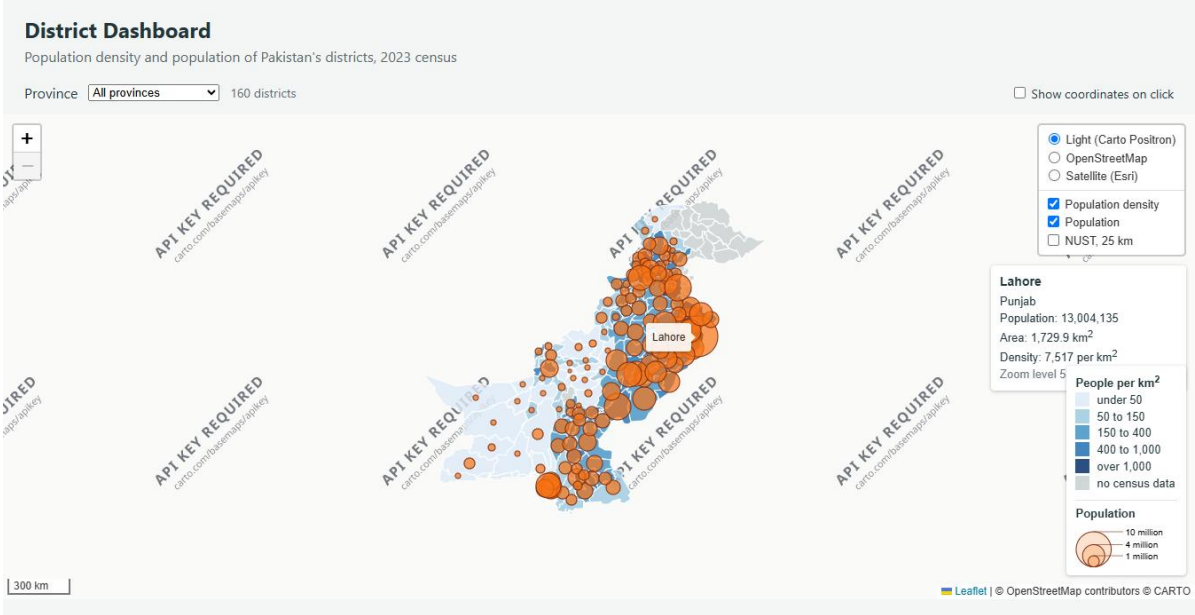


Figure 5 (Checkpoint 5): Hover highlight on Lahore showing dark outline, tooltip, and info panel reporting population (13,004,135) and density (7,339 per km²).

6. Proportional Symbols and Custom Panes (Checkpoint 6)

Proportional symbols represent counts (pop_2023). To prevent polygon hover bringToFront() from occluding circle symbols, a custom Leaflet map pane ('symbols', z-index 450) was established above overlayPane (400) and below markers/tooltips (600+).

Perceptual Sizing: Radius is computed using the square root transformation (radius = sqrt(pop_2023) / 160) so that symbol visual area—not radius—scales linearly with population.

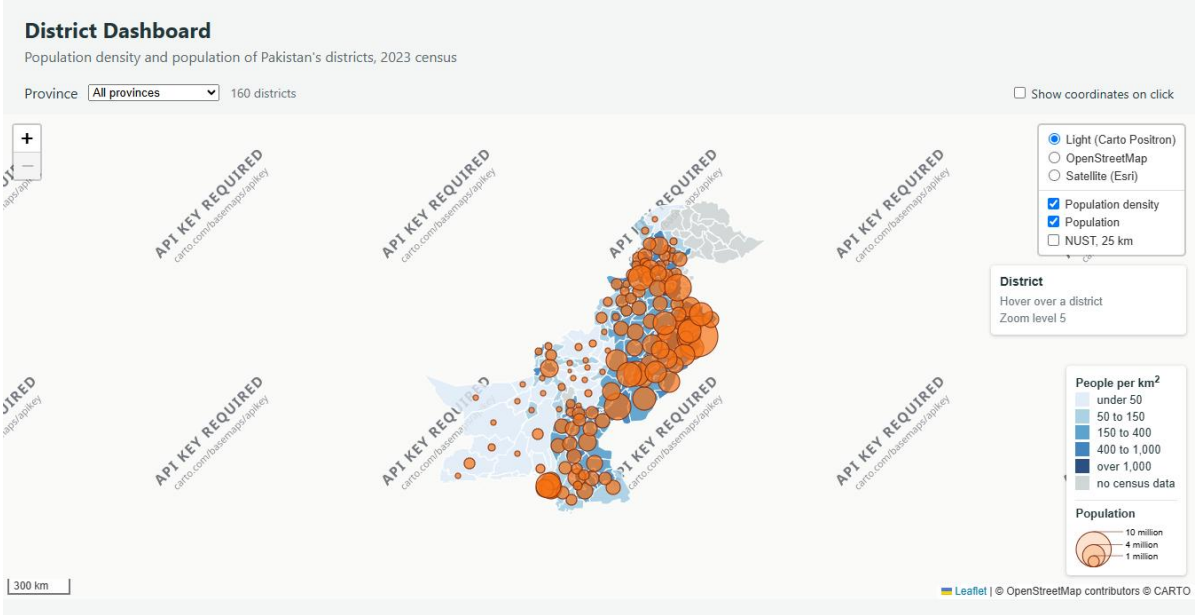


Figure 6A (Checkpoint 6): Complete dashboard displaying proportional population circles floating in their dedicated pane above density shading, with dual legend.

Metres vs. Pixels Demonstration (NUST Overlay)

The NUST overlay directly contrasts L.circle (radius: 25000 in ground metres) against L.circleMarker (radius: 6 in screen pixels). At Zoom 6 (national scale), the 25 km circle appears compact; at Zoom 10 (metropolitan scale), it spans the screen while precisely matching 25 km on the scale bar. In contrast, L.circleMarker maintains a constant 6-pixel visual footprint.

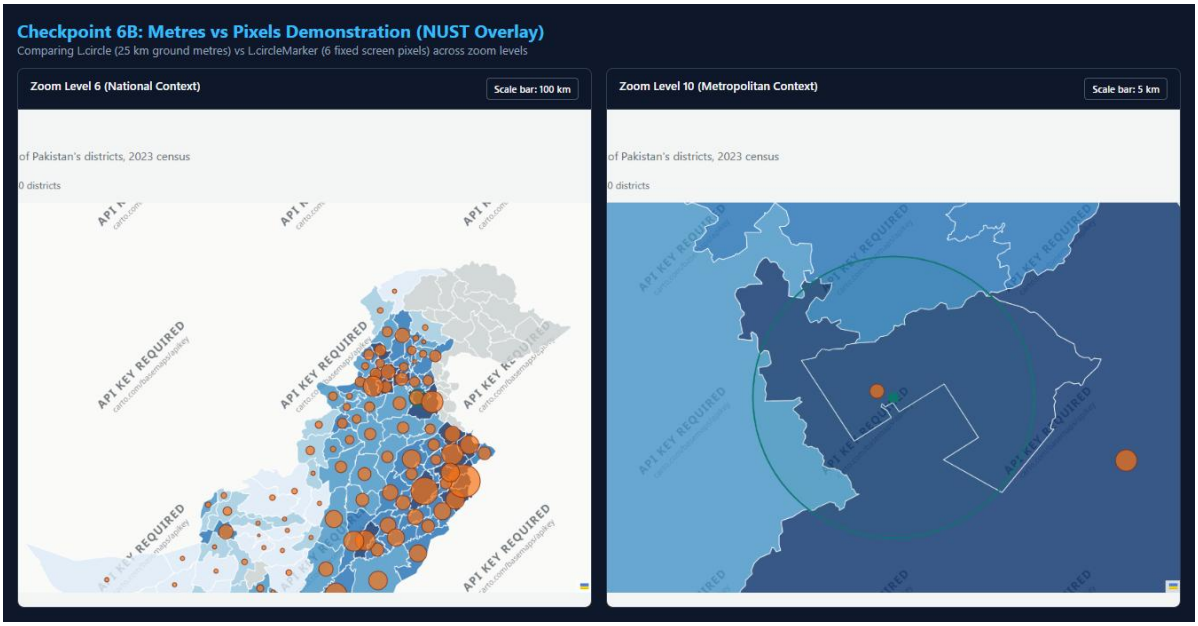


Figure 6B (Checkpoint 6): NUST overlay at Zoom 6 (scale bar 100 km) vs Zoom 10 (scale bar 5 km) illustrating ground distance vs screen pixel rendering.

7. Map Events: Dynamic Registration and Removal (Checkpoint 7)

An interactive checkbox (#coords) toggles a coordinate click handler (map.on / map.off).

Laboratory Observations:

1. Sea Click & District Click: Clicking the Arabian Sea opens a coordinate popup (24.2185°N, 66.8421°E). Clicking a district executes both handlers: the district click event zooms to the district bounds, while the map click event opens the coordinate popup.
2. Click Propagation: Clicking inside the legend produces no popup because L.DomEvent.disableClickPropagation(div) halts bubbling. When removed, clicks on the legend bleed through to the map canvas beneath.
3. Unticking: Unchecking #coords invokes map.off('click', showCoordinates), cleanly deregistering the listener.

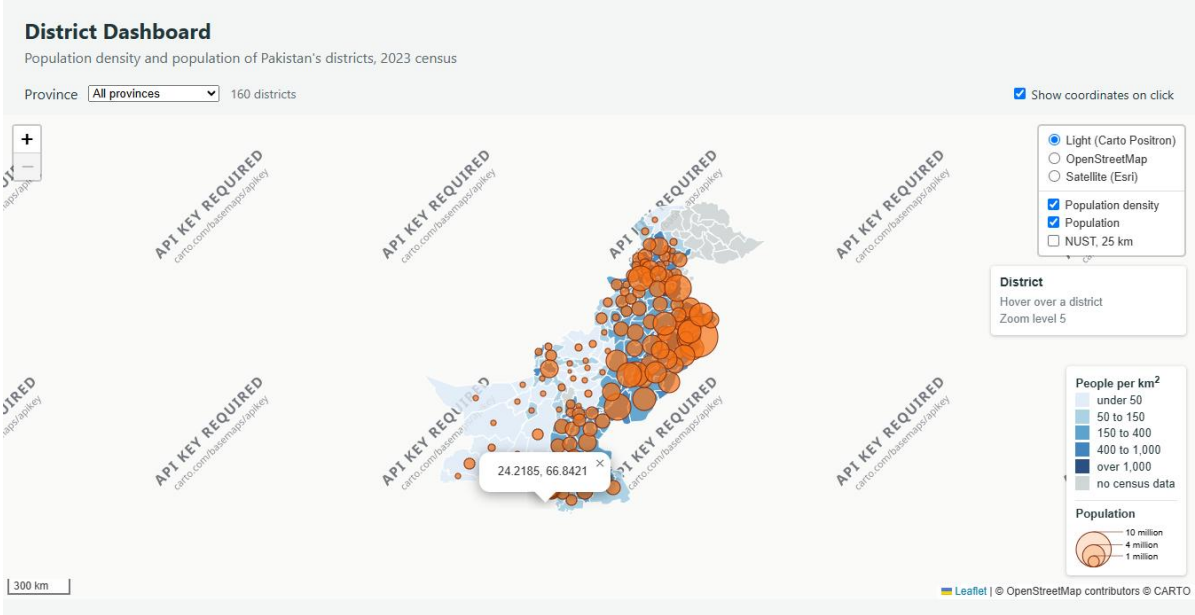


Figure 7 (Checkpoint 7): Active coordinate popup in the Arabian Sea south of Karachi.

8. Filter by Province Extension (Checkpoint 8)

A dynamic dropdown populated from district feature attributes filters the map by province. When Gilgit Baltistan is selected, 14 districts are reported, shaded grey, and 0 circles appear. This occurs because 2023 census counts are null for this region; the hasPopulation filter safely drops null features to prevent NaN runtime exceptions.

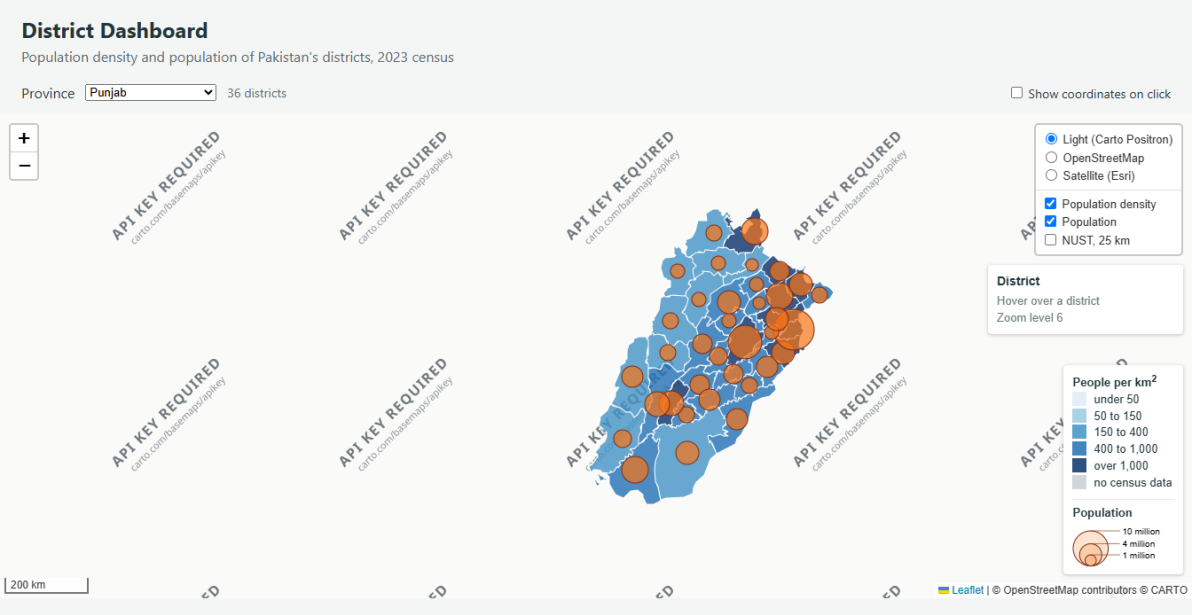


Figure 8 (Checkpoint 8): Filter active for Punjab (41 districts) with layer overlays switchable on the rebuilt layers.

9. SVG vs Canvas Rendering Benchmark (Checkpoint 9)

The rendering engine was benchmarked across 5 reloads using performance.now() via the URL parameter index.html?canvas.

Metric / Architectural Feature	SVG Mode (Default)	Canvas Mode (?canvas)
draw('all'), Median of 5 runs	48.1 ms (77.6, 46.9, 69.0, 48.1, 47.0)	46.5 ms (35.2, 47.6, 48.1, 46.5, 28.8)
Elements inside leaflet-overlay-pane	162 elements (1 <svg> + 1 <g> + 160 <path>)	1 element (1 <canvas>)
Elements inside leaflet-symbols-pane	137 elements (1 <svg> + 1 <g> + 135 <path>)	1 element (1 <canvas>)
Hover highlight still works?	Yes (Native SVG event listeners)	Yes (Leaflet internal hit detection)
Can you inspect one district in Elements?	Yes (Discrete DOM path nodes)	No (Flat bitmap; only <canvas> tag)

Architectural Analysis: For 160 districts and 135 circles, the performance difference is negligible (~48 ms vs ~46 ms). However, for the tehsil layer (administrative level 3, ~600 units with >100,000 vertices), SVG suffers from severe DOM bloat, memory footprint growth (>150 MB), and reflow stutter during panning.

Canvas excels at scale because it renders directly to an immediate-mode 2D pixel buffer, bypassing the DOM entirely and maintaining smooth 60 FPS interactions.

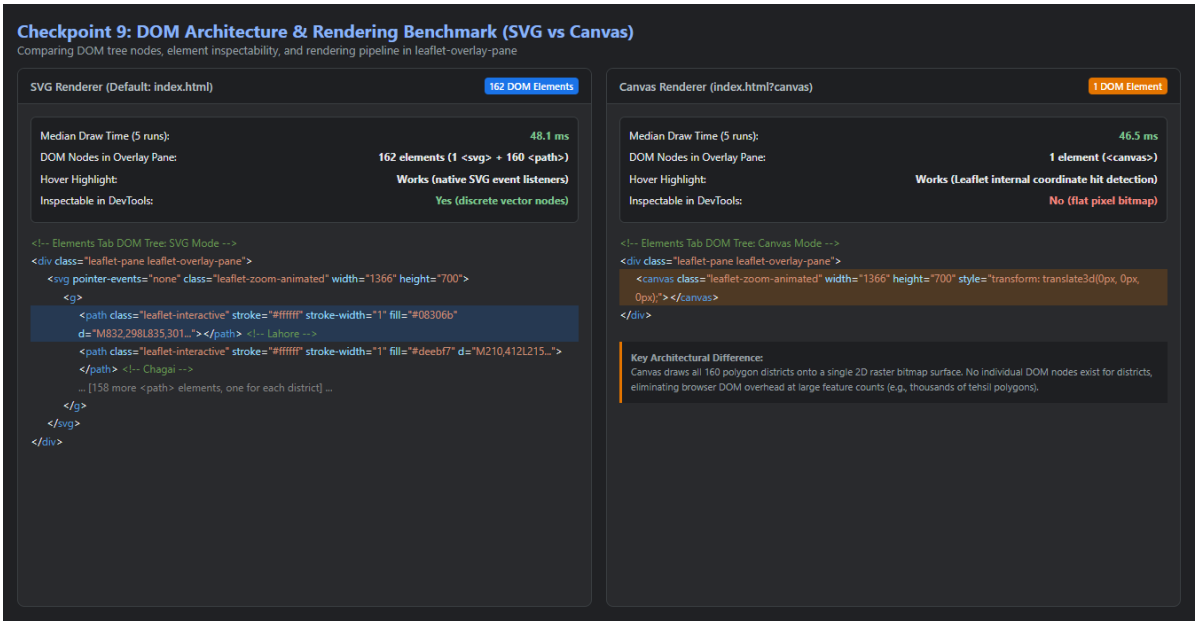


Figure 9 (Checkpoint 9): Elements tab comparison showing 162 discrete SVG path nodes in default mode vs. a single <canvas> element in Canvas mode.

10. Answers to Laboratory Questions (Section 10.3)

Question 1: Large vs. Small Districts on a Count Choropleth

Q: Lab 2 shaded districts by population. Name one large, thinly populated district and one small, crowded one, and explain how each would mislead on the Lab 2 map.

Answer: Chagai in Balochistan (45,912 km², pop ~269,000, density ~5.8/km²) is large and thinly populated. Lahore in Punjab (1,772 km², pop ~13,004,000, density ~7,339/km²) or Karachi Central in Sindh (69 km², pop ~3,822,000, density ~55,391/km²) are small and crowded.
On the Lab 2 map, Chagai occupied a massive portion of the canvas and appeared visually prominent simply because of its geographic area, falsely suggesting immense population concentration. In contrast, Lahore and Karachi Central appeared as tiny, barely noticeable specks despite housing millions. Choropleths shade areas, so the displayed metric must be normalized by area (density) to prevent area from biasing the reader's perception.

Question 2: Perceptual Sizing with Square Roots

Q: radiusFor takes a square root. What would the map suggest about Lahore against a district of 3 million if it did not?

Answer: The human eye assesses circular symbols by area ($A = \pi r^2$), not radius. If radius scaled linearly with population, the radius ratio between Lahore (13M) and a 3M district would be 13/3 ≈ 4.33. The resulting visual area ratio would be $(4.33)^2 \approx 18.78$. The map would visually suggest that Lahore is nearly 19 times larger than a 3-million district rather than 4.3 times, severely misleading the reader and occluding neighboring districts. Taking the square root ensures perceived symbol area scales 1:1 with population.

Question 3: Persistent Layer Groups in Layer Controls

Q: Why does the layer control hold districtGroup rather than the L.geoJSON layer itself?

Answer: During province filtering, the district GeoJSON layer is destroyed and re-instantiated. If the layer control held a direct reference to the initial L.geoJSON layer, its switcher checkbox would become orphaned, pointing to an object no longer present on the map. By holding a permanent L.featureGroup() (districtGroup), the layer control maintains an unbroken reference while draw() clears and updates its internal layers.

Question 4: Dedicated Panes vs OverlayPane Draw Order

Q: What would go wrong if the circles were drawn in overlayPane instead of the symbols pane? Describe what the user would see when hovering.

Answer: Both polygons and circle markers are vector SVG paths. Within a shared overlayPane (z-index 400), invoking e.target.bringToFront() during district hover moves the hovered polygon's SVG <path> to the bottom of the container, drawing it in front of all other paths. If circles were in overlayPane, hovering over a district would lift the large colored polygon above the proportional circles, hiding the circles beneath the polygon fill. The symbols pane (z-index 450) ensures circles remain permanently on top.

Question 5: Named Functions and map.off

Q: Why can map.off remove showCoordinates, when it could not remove an anonymous function passed to map.on?

Answer: In JavaScript, functions are compared by object reference (===). Leaflet stores listeners in an internal array. map.off looks up the exact function reference to remove it. When a named function (showCoordinates) is used, the exact memory reference is provided. If an anonymous function is used (function(e) { ... }), there is no persistent reference handle. Calling map.off with another inline function creates a different object that fails the equality check, leaving the listener permanently bound.

Question 6: Ground Metres vs. Screen Pixels on the NUST Overlay

Q: On the NUST overlay, which circle would you use to show a 25 km evacuation zone, and which to mark the campus on a national map? Why?

Answer: For a 25 km evacuation zone, use L.circle (radius: 25000 in ground metres). An evacuation zone is a physical real-world spatial boundary; as the user zooms in or out, L.circle expands or contracts on

screen to constantly represent 25 kilometers of ground terrain, aligning with the scale bar.
To mark the campus on a national map, use L.circleMarker (radius: 6 in screen pixels). A map marker identifies a location point and must remain legible across all zoom levels without expanding into an oversized blob at high zoom or vanishing into sub-pixel obscurity at national zoom.